

DS 203 – Big Data Essentials

Advanced SQL Lab 2 – Indexes

Marks: 100

Student ID: 2025014777

Student Name: Tetsuya Maeda

Answer the following questions:

Q1. Create an index on the 'email' column in 'customer'.

Query:

```
CREATE INDEX idx_customer_email ON customer(email);
```

Output Screenshot:

```
hint: You can drop constraint customer_pkey on table customer instead.
dvdrental=# \i /Users/tetsuya/Develop/dataScience/data-science-python-practice/DS203/003.Lab2/question1.sql
CREATE INDEX
dvdrental=# SELECT *FROM pg_indexes WHERE tablename = 'customer';

```

schemaname	tablename	indexname	tablespace	indexdef
public	customer	customer_pkey		CREATE UNIQUE INDEX customer_pkey ON public.customer USING btree (customer_id)
public	customer	idx_customer_email		CREATE INDEX idx_customer_email ON public.customer USING btree (email)

```
(2 rows)
```

Q2. Drop the index created in Q1 and observe how performance changes. Look for impact on execution time and planning time.

Query 1:

First, before dropping the index write the query to extract information from customer table based on email use EXPLAIN ANALYZE command to show execution and planning times:

```
EXPLAIN ANALYZE SELECT * FROM customer WHERE email = 'some.email@example.com';
```

Output Screenshot:

```
dvdrental=# EXPLAIN ANALYZE SELECT * FROM customer WHERE email = 'some.email@example.com';

```

QUERY PLAN
Index Scan using idx_customer_email on customer (cost=0.28..8.29 rows=1 width=70) (actual time=0.685..0.686 rows=0.00 loops=1)
Index Cond: ((email)::text = 'some.email@example.com'::text)
Index Searches: 1
Buffers: shared read=2
Planning:
Buffers: shared hit=23 read=1 dirtied=1
Planning Time: 1.310 ms
Execution Time: 0.712 ms

```
(8 rows)
```

Query 2:

Second, drop the index:

```
DROP INDEX idx_customer_email;
```

Output Screenshot:

```
dvdrental=# DROP INDEX idx_customer_email;
DROP INDEX
dvdrental=# SELECT *FROM pg_indexes WHERE tablename = 'customer';
 schemaname | tablename | indexname | tablespace | indexdef
-----
 public     | customer | customer_pkey |          | CREATE UNIQUE INDEX customer_pkey ON public.customer USING btree (customer_id)
(1 row)
```

Query 3:

Third, after dropping the index write the query to extract information from customer table based on email again and use EXPLAIN ANALYZE command to show execution and planning times after dropping index:

Output Screenshot:

```
dvdrental=# EXPLAIN ANALYZE SELECT * FROM customer WHERE email = 'some.email@example.com';
              QUERY PLAN
-----
Seq Scan on customer  (cost=0.00..16.49 rows=1 width=70) (actual time=0.226..0.227 rows=0.00 loops=1)
  Filter: ((email)::text = 'some.email@example.com'::text)
  Rows Removed by Filter: 599
  Buffers: shared hit=9
Planning:
  Buffers: shared hit=6
Planning Time: 1.559 ms
Execution Time: 0.289 ms
(8 rows)
```

Q3. Create a compound index on 'first_name, last_name' in the actor table

Query:

```
CREATE INDEX idx_actor_first_last_name ON actor(first_name, last_name);
```

Output Screenshot:

```
dvdrntal=# SELECT *FROM pg_indexes WHERE tablename = 'actor';
 schemaname | tablename |      indexname      | tablespace |          indexdef          |
-----|-----|-----|-----|-----|
 public     | actor    | actor_pkey          |             | CREATE UNIQUE INDEX actor_pkey ON public.actor USING btree (actor_id) |
 public     | actor    | idx_actor_first_last_name |             | CREATE INDEX idx_actor_first_last_name ON public.actor USING btree (first_name, last_name) |
(2 rows)
```

Q4. Use pg_indexes to list all indexes on the 'actor' table.

Query:

```
SELECT * FROM pg_indexes WHERE tablename = 'actor';
```

Output Screenshot:

```
dvdrntal=# SELECT * FROM pg_indexes WHERE tablename = 'actor';
 schemaname | tablename |      indexname      | tablespace |          indexdef          |
-----|-----|-----|-----|-----|
 public     | actor    | actor_pkey          |             | CREATE UNIQUE INDEX actor_pkey ON public.actor USING btree (actor_id) |
 public     | actor    | idx_actor_first_last_name |             | CREATE INDEX idx_actor_first_last_name ON public.actor USING btree (first_name, last_name) |
(2 rows)
```

Q5. Try indexing a column with few unique values like “active” column in “customers” table and compare the performance - does it help? Explain your answer.

Answer:

Indexing a column with low cardinality like the ‘active’ column in the customer table is generally not recommended and often does not improve performance. Because

indexing is useful for high selectivity columns. But data type of “active” column is boolean, if we create indexes it does not help to select each row.

Marking Criteria:

For each question from Q1-Q4:

Correct Query = 10 Marks

Correct Output Screenshot = 10 Marks

For Q5: 20 marks.

Total Marks: 20 x 5 (questions) = 100 Marks.

Submission Instructions:

Submit the following:

1. Convert this document into PDF and rename it as “**DS 203 - Lab 2 – StudentName.pdf**” then submit in the appropriate submission on Google Classroom.
2. Also, submit the .sql script containing queries for Q1-Q4 in single file. You can name the script file as “**DS 203 - Lab 2 – StudentID.sql**”.